

PREDIKSI PENDAPATAN FILM MENGGUNAKAN METODE RANDOM FOREST REGRESSION



Dosen Pengampu Mata Kuliah:

Riskyana Dewi Intan Puspitasari, M.Kom.

Ike Fitriyaningsih, M.Si

Disusun Oleh:

- | | |
|---------------------------------|-------------|
| 1. Ananda Keissa Az Zahra | 24031554051 |
| 2. Laili Nurrohmatul Fadhila Z. | 24031554093 |
| 3. Eka Putri Maharani | 24031554121 |

**PROGRAM STUDI SAINS DATA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA
2025**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Industri film terus berkembang pesat dan semakin kompetitif dari tahun ke tahun. Jumlah produksi film yang meningkat membuat para produser, distributor, dan platform digital harus mampu mengambil keputusan yang lebih strategis sebelum sebuah film dirilis. Dalam proses tersebut, prediksi terhadap potensi keberhasilan komersial film menjadi salah satu aspek penting yang dapat membantu menentukan langkah promosi, strategi distribusi, hingga penentuan anggaran produksi. Penelitian terdahulu menunjukkan bahwa data terkait karakteristik film seperti genre, budget, popularitas, durasi, serta penilaian penonton dapat digunakan untuk memprediksi performa finansial sebuah film dengan tingkat akurasi yang cukup baik (Ni et al., 2022). Dengan kata lain, pendekatan berbasis data kini menjadi alat yang relevan dalam mendukung pengambilan keputusan di industri hiburan.

Di sisi lain, perkembangan teknik machine learning memberikan peluang lebih luas untuk menganalisis faktor-faktor yang memengaruhi pendapatan film. Studi oleh Lee, KC & Choeh (2020) menunjukkan bahwa metode berbasis ensemble, seperti Random Forest, memberikan prediksi pendapatan film yang lebih akurat dibandingkan regresi linear atau metode sederhana lainnya. Berdasarkan latar belakang tersebut, metode Random Forest dipandang relevan dan potensial untuk digunakan dalam studi prediksi revenue film, karena mampu menangani kombinasi fitur numerik dan kategorikal, toleran terhadap outlier dan distribusi data yang tidak normal (misalnya budget & revenue) dan bisa memberikan output prediksi numerik konkret yang bisa dijadikan acuan bagi produser atau investor film untuk mengambil keputusan sebelum produksi atau distribusi.

1.2 Rumusan Masalah

1. Faktor apa saja yang berpengaruh terhadap pendapatan film?
2. Bagaimana proses preprocessing dan feature engineering dapat mempersiapkan data film agar sesuai untuk model Random Forest Regressor?
3. Seberapa baik algoritma Random Forest Regressor dalam memprediksi revenue film berdasarkan dataset yang diperoleh?
4. Model seperti apa yang dihasilkan, dan apakah model tersebut memiliki tingkat akurasi yang cukup untuk digunakan sebagai bahan estimasi kesuksesan finansial film?

1.3 Tujuan

1. Mengidentifikasi variabel-variabel yang memiliki pengaruh signifikan terhadap revenue film.
2. Melakukan preprocessing dan feature engineering, termasuk one-hot encoding untuk variabel kategorikal.

3. Menerapkan algoritma Random Forest Regressor untuk memprediksi pendapatan film berdasarkan dataset TMDB.
4. Mengevaluasi performa model menggunakan regresi seperti MAE, MSE, RMSE, dan R^2 .

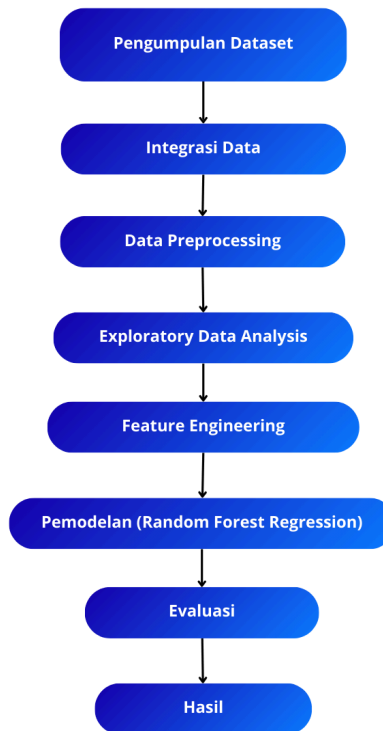
1.4 Manfaat

1. Memberikan gambaran kepada pihak industri mengenai faktor-faktor yang memengaruhi pendapatan film.
2. Menjadi contoh penerapan machine learning, terutama random forest regression dalam bidang industri kreatif.
3. Menyediakan model prediksi yang dapat digunakan untuk estimasi performa komersial film.

BAB II

METODOLOGI PROYEK

2.1 Diagram Alur



2.2 Tahapan Pengerjaan

2.2.1 Pengumpulan Dataset

Penelitian ini menggunakan subset data yang bersumber dari [TMDB 5000 Movie Dataset](#), dilengkapi dengan data ulasan (*review*) penonton dari sumber [Rotten Tomatoes movies and critic review](#). Untuk mengoptimalkan kinerja model prediksi, dilakukan seleksi atribut (*feature selection*) sehingga hanya variabel-variabel yang relevan secara bisnis dan statistik yang dipertahankan. Dataset final terdiri dari kolom-kolom berikut:

1. Dataset TMDB 5000 Movie Title
 - a. `movie_title`: Judul resmi film. Digunakan sebagai identifier utama untuk menggabungkan data antar dataset.
 - b. `revenue`: Total pendapatan kotor (gross revenue) film dalam Dolar AS. Variabel ini merupakan Variabel Target (Label) yang akan diprediksi oleh model.
 - c. `budget`: Modal produksi film (USD)
 - d. `popularity`: Tingkat popularitas film berdasarkan metrik TMDB
 - e. `vote_count` : Jumlah user yang memberikan rating. Indikator hype penonton.

- f. `vote_average`: Nilai rata-rata rating yang diberikan penonton (skala 0-10). Merepresentasikan persepsi kualitas film secara umum.
2. Dataset Rotten Tomatoes movies and critic review
- a. `rotten_tomatoes_link`: Tautan unik halaman film. Berfungsi sebagai *unique identifier* (ID) untuk memastikan tidak ada duplikasi data ulasan.
 - b. `title`: Judul film yang digunakan sebagai kunci pencocokan (*key*) saat proses *merging* dengan dataset TMDb.
 - c. `content_rating`: Klasifikasi rating usia penonton (misal: PG-13, R, G). Menentukan target demografi pasar film.
 - d. `genres`: Kategori genre film (misal: Action, Horror). Fitur krusial untuk melihat tren pasar.
 - e. `directors`: Nama sutradara film. Digunakan untuk menganalisis pengaruh reputasi sutradara terhadap pendapatan.
 - f. `authors`: Nama penulis naskah
 - g. `actors`: Daftar penonton utama. Digunakan untuk mengukur daya tarik aktor dalam mendatangkan penonton
 - h. `original_release_date`: Tanggal rilis perdana di bioskop. Diekstraksi menjadi fitur Tahun dan Bulan untuk melihat pola musiman
 - i. `runtime`: Durasi tayang film dalam satuan menit.
 - j. `production_company`: Studio atau perusahaan produksi. Mengindikasikan besarnya kekuatan distribusi dan pemasaran film.
 - k. `review_content`: Isi ulasan lengkap dari kritikus. Data ini diproses menggunakan TF-IDF untuk mengekstraksi kata kunci (*keywords*) dominan yang mempengaruhi persepsi penonton.
 - l. `review_type`: Label sentimen dari ulasan, yaitu Fresh (Positif) atau Rotten (Negatif). Digunakan untuk menghitung rasio kualitas film.

2.2.2 Persiapan Data: Integrasi Data

Mengingat sumber dari kedua dataset ini berbeda, maka diperlukan tahapan integrasi untuk menyelaraskan struktur data.

```
import pandas as pd

# read file
movies = pd.read_csv('/content/dataset-pemteks/rotten_tomatoes_movies.csv')
review = pd.read_csv('/content/dataset-pemteks/rotten_tomatoes_critic_reviews.csv')
revenue = pd.read_csv('/content/dataset-pemteks/tmdb_5000_movies.csv')

# rename
revenue.rename(columns={'title': 'movie_title'}, inplace=True)

# subset
selected_movies = movies[['rotten_tomatoes_link', 'movie_title', 'content_rating', 'genres', 'directors', 'authors', 'actors',
                           'original_release_date', 'runtime', 'production_company']]
selected_review = review[['rotten_tomatoes_link', 'review_content', 'review_type']]
selected_revenue = revenue[['movie_title', 'revenue', 'popularity', 'budget', 'vote_count', 'vote_average']]

# merge
df1 = pd.merge(selected_movies, selected_review, on='rotten_tomatoes_link')
df = pd.merge(df1, selected_revenue, on='movie_title')
```

```

review_counts = df.groupby('movie_title').size().reset_index(name='jumlah_review')

film_layak = review_counts[review_counts['jumlah_review'] >= 30]['movie_title']
df_filtered = df[df['movie_title'].isin(film_layak)].copy()

df_30 = df_filtered.groupby('movie_title').head(30)

final_counts = pd.crosstab(df_30['movie_title'], df_30['review_type'])

review_counts = df.groupby('rotten_tomatoes_link').size().reset_index(name='review_content_count')
filtered_links = review_counts[review_counts['review_content_count'] >= 30]['rotten_tomatoes_link']
df = df[df['rotten_tomatoes_link'].isin(filtered_links)]
df = df.groupby('rotten_tomatoes_link').head(30)

df_aggregated = df.groupby('rotten_tomatoes_link')['review_content'].apply(lambda x: ' '.join(x.dropna())).reset_index()
df = df.drop(columns=['review_content'])
df = pd.merge(df, df_aggregated, on='rotten_tomatoes_link', how='left')
df = df.drop_duplicates(subset=['rotten_tomatoes_link'])

df = pd.merge(df, final_counts, on='movie_title')

```

Tahapan-tahapan Integrasi Data:

1. Subsetting Data

Pada tahap ini, kami mengambil kolom-kolom yang relevan dengan tujuan penelitian, seperti revenue, budget, popularity, genres, production_company.

2. Penggabungan Data

Sumber Rotten Tomatoes movies and critic review terdapat dua dataset yang perlu digabungkan berdasarkan kolom rotten_tomatoes_link. Metode penggabungan yang digunakan adalah Left Join, di mana data metadata film dipertahankan sebagai basis utama, dan data ulasan ditambahkan pada film yang bersesuaian. Hal ini memastikan bahwa setiap entri film memiliki kelengkapan data atribut (seperti budget dan genre) sekaligus data teks ulasan. Selanjutnya, dilakukan penggabungan juga dengan Dataset TMDB 5000 Movie Title menggunakan atribut movie_title sebagai kunci pencocokan (key) untuk mendapatkan informasi tambahan yang tidak tersedia di Rotten Tomatoes, seperti data popularitas harian (popularity) dan jumlah partisipasi penonton (vote count).

3. Penyaringan Data

Untuk menjaga validitas analisis teks dan menghindari bias akibat sampel yang terlalu sedikit, diterapkan mekanisme penyaringan (*thresholding*) sebagai berikut:

- Minimal Ulasan:** Hanya film yang memiliki minimal 30 ulasan yang diikutsertakan dalam dataset akhir. Film dengan ulasan di bawah ambang batas ini dieliminasi karena dianggap tidak merepresentasikan konsensus penonton yang cukup signifikan untuk dianalisis.
- Pembatasan Sampel:** Untuk menjaga keseimbangan komputasi, diambil sampel acak sebanyak maksimal 30 ulasan per film.

4. Agregasi Ulasan

Karena dalam satu film memiliki banyak ulasan, maka kami melakukan:

- Agregasi Teks:** Seluruh teks ulasan individu pada satu film digabungkan (*concatenated*) menjadi satu dokumen teks panjang yang nantinya akan menjadi input untuk proses ekstraksi fitur TF-IDF

- b. Agregasi Sentimen: Dilakukan perhitungan distribusi label sentimen (jumlah label 'Fresh' dan 'Rotten') untuk setiap film. Dari agregasi ini, diturunkan fitur baru berupa Freshness Ratio. Fitur ini berfungsi sebagai indikator kualitas film yang ternormalisasi, sehingga penilaian kualitas tidak bias terhadap jumlah ulasan (popularitas) dan memberikan skala penilaian yang lebih presisi (0.0 hingga 1.0) bagi model regresi.

2.2.3 Data Preprocessing

Pada tahap pra-pemrosesan data, dilakukan pembersihan data untuk meningkatkan kualitas input model. Data review yang bersumber dari Rotten Tomatoes movies and critic review perlu dikonversi menjadi format yang terstruktur.

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer

try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    nltk.download('stopwords', quiet=True)

stop_words = set(stopwords.words('english'))
manual_stopwords = {
    "movie", "film", "movies", "films",
    "scene", "scenes", "character", "characters",
    "story", "plot", "really", "very", "much", "well", "quite", "just",
    "even", "still", "also", "though", "rather",
    "little", "lot", "thing", "things", "something",
    "anything", "everything", "nothing",
    "one", "two", "three", "many", "few",
    "bit", "kind", "sort", "type",
    "someone", "anyone", "everyone", "people",
    "fun", "good", "great", "nice", "bad",
    "better", "worst", "cool",
    "interesting",
    "get", "got", "give", "take",
    "make", "made", "makes",
    "seems", "seem", "seemed",
    "something", "anything", "everything", "nothing",
    "hollywood", "disney", "cast", "acting",
    "actor", "actors", "actress", "performance",
    "time", "minute", "way", "part", "end", "start",
    "they", "them", "their", "theirs",
    "he", "him", "his",
    "she", "her", "hers",
    "we", "us", "our", "ours",
    "you", "your", "yours",
```

```

    "feel", "felt", "feels",
    "look", "looks", "looked",
    "try", "tries", "tried",
    "could", "may", "might", "would", "will", "can",
    "ever", "never", "always", "first", "last", "new", "old", "year", "time", "moment",
    "big", "high", "low", "long", "short", "real", "fake",
    "come", "go", "going", "turn", "without", "with", "work", "working",
    "see", "watch", "watching", "seen", "look", "looking",
    "director", "direct", "filmmaker", "screen", "script", "cast", "actor", "actress",
    "performance", "played", "play", "star", "audience", "people", "many", "much",
    "like", "really", "well", "good", "bad", "great", "best", "better",
    "another", "every", "find", "man", "picture", "set", "want",
    "direct", "enough", "hard", "life", "tale", "coming", "getting"
}
stop_words.update(manual_stopwords)
stemmer = SnowballStemmer("english")

def clean_text(row):
    text = str(row['review_content'])
    title = str(row['movie_title']).lower()

    if pd.isna(text): return ""

    # 1. Lowercase
    text = text.lower()

    # 2. Hapus Judul Film didalam review
    title_clean = re.sub(r'[^a-z\s]', ' ', title)
    title_words = set([w for w in title_clean.split() if len(w) > 3])

    # 3. Hapus Angka & Tanda Baca
    text = re.sub(r'\d+', ' ', text)
    text = re.sub(r'[^a-z\s]', ' ', text)

    # 4. Tokenisasi
    words = text.split()

    clean_words = []
    for w in words:
        # Hapus Judul
        if w in title_words: continue

        # Hapus Stopwords
        if w not in stop_words and len(w) > 2:
            stem = stemmer.stem(w)
            clean_words.append(stem)

    return " ".join(clean_words)

df['review_clean_stemmed'] = df.apply(clean_text, axis=1)

```

Tahapan-tahapan *Data Preprocessing*:

1. Case Folding

Mengubah seluruh huruf dalam teks review menjadi huruf kecil (*lowercase*) untuk menjaga konsistensi data dan menghindari duplikasi fitur akibat perbedaan kapitalisasi.

2. Penghapusan Judul Film yang disebutkan di Review

Menghapus nama judul film yang terdapat di dalam teks ulasan masing-masing baris untuk memastikan model mempelajari sentimen dari kata-kata deskriptif

3. Pembersihan Tanda Baca

Menghilangkan karakter tanda baca, angka, dan simbol khusus. Tanda baca digantikan dengan spasi untuk mencegah penggabungan kata yang tidak disengaja.

4. Tokenisasi

Memecah kalimat ulasan menjadi potongan-potongan kata tunggal

5. Stopword Removal

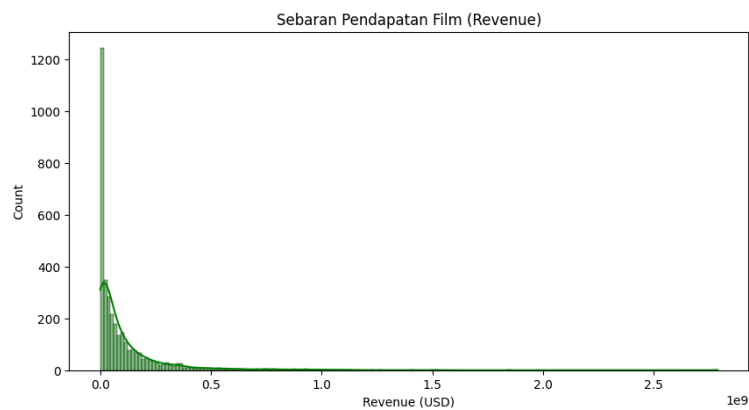
Mengeliminasi kata-kata penghubung umum (seperti *and*, *or*, *the*) serta kata-kata spesifik domain yang terlalu sering muncul (seperti *movie*, *film*) karena dianggap tidak

memberikan kontribusi informasi yang signifikan terhadap klasifikasi sentimen atau pendapatan.

6. Stemming

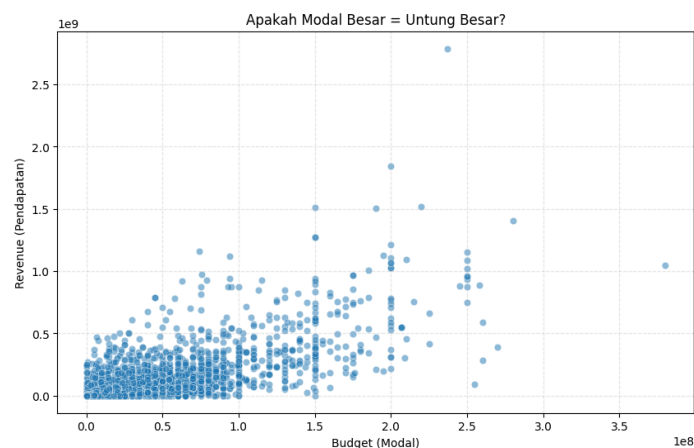
Mentransformasi kata ke dalam bentuk dasarnya (misalnya mengubah '*acting*' dan '*acted*' menjadi '*act*'). Proses ini bertujuan untuk menyatukan kata-kata yang memiliki makna serupa.

2.2.4 Exploratory Data Analysis



Gambar 3.1: Distribusi Pendapatan Film (Revenue)

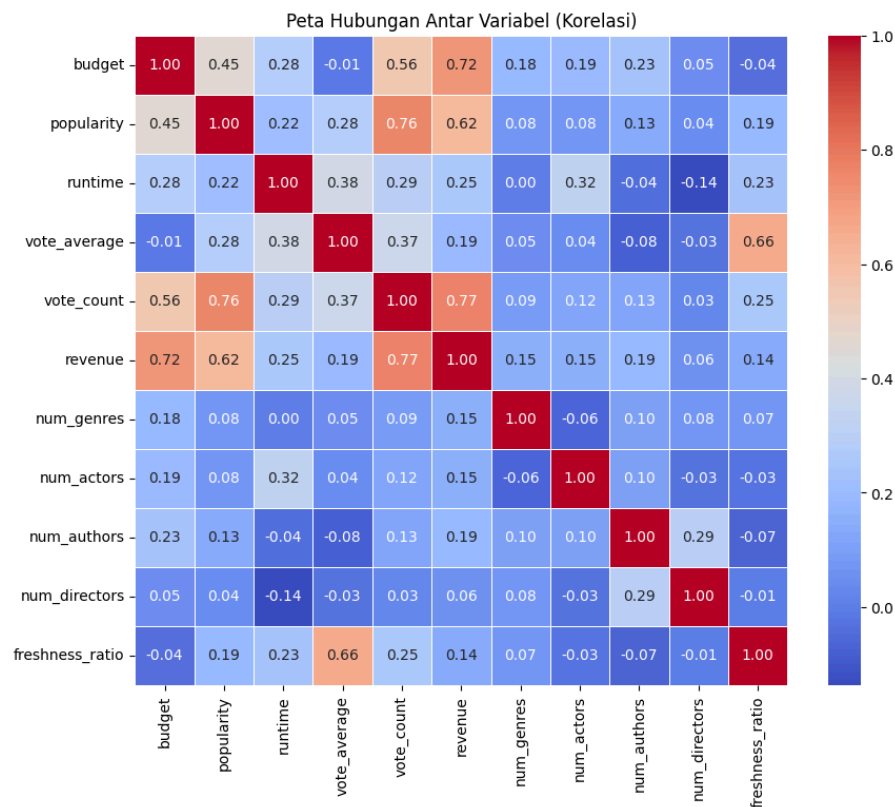
Berdasarkan Gambar 3.1, terlihat bahwa distribusi pendapatan film memiliki kemiringan positif (*right-skewed*). Sebagian besar film memiliki pendapatan di bawah 100 juta dolar, sementara terdapat sejumlah kecil film (outliers) dengan pendapatan ekstrem di atas 1 miliar dolar. Hal ini mengindikasikan perlunya transformasi data (seperti Log Transformation) agar model tidak bias terhadap nilai ekstrem tersebut.



Gambar 3.2: Hubungan antara budget dan revenue film

Grafik scatter pada Gambar 3.2 menunjukkan bahwa film dengan budget lebih besar cenderung menghasilkan pendapatan yang lebih tinggi. Namun, sebaran titik yang sangat luas

menunjukkan bahwa hubungan ini tidak kuat, karena banyak film berbudget besar yang tidak menghasilkan revenue tinggi, dan beberapa film yang berbudget kecil justru mampu meraih keuntungan besar. Dengan demikian, modal besar dapat meningkatkan peluang pendapatan tinggi, namun bukan jaminan kesuksesan sebuah film.



Gambar 3.3: Matriks Korelasi Fitur Numerik

Heatmap pada Gambar 3.3 ini memperlihatkan korelasi antar variabel numerik dalam dataset film. Secara keseluruhan, heatmap menunjukkan bahwa hanya beberapa variabel yang memiliki hubungan kuat dengan pendapatan, yaitu budget, vote_count, dan popularity. Sementara variabel lain seperti runtime, num_genres, num_actors, num_authors, dan num_directors memiliki korelasi lemah sehingga pengaruhnya relatif kecil secara linear. Heatmap ini membantu mengidentifikasi fitur-fitur yang paling relevan dan menjadi dasar dalam pemilihan fitur untuk membangun model prediksi revenue.

2.2.5 Feature Engineering

1. TF-IDF dan LSA

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import PCA
import pandas as pd

tfidf = TfidfVectorizer(max_features=1000, min_df=3)
tfidf_matrix = tfidf.fit_transform(df['review_clean_stemmed'])

# n = topic
n_components = 10
pca = PCA(n_components=n_components)
pca_matrix = pca.fit_transform(tfidf_matrix.toarray())

# membuat nama kolom topic 1 - 10
pca_cols = [f"Text_Topic_{i+1}" for i in range(n_components)]
df_pca = pd.DataFrame(pca_matrix, columns=pca_cols)

df.reset_index(drop=True, inplace=True)
df_pca.reset_index(drop=True, inplace=True)
tfidf_df = pd.concat([df, df_pca], axis=1)
tfidf_df = tfidf_df.drop(columns=['review_clean_stemmed', 'review'], errors='ignore')

```

```

import pandas as pd

print("ISI TOPIK PCA")

vocab = tfidf.get_feature_names_out()

for i in range(n_components):
    topic = pca.components_[i]
    top_word_indices = topic.argsort()[::-1]
    top_words = [vocab[idx] for idx in top_word_indices]

    print(f"\nTopik {i+1} berisi tentang: {' '.join(top_words)}")

```

ISI TOPIK PCA

Topik 1 berisi tentang: comedi, funni, laugh, joke, romant, humor, gag, charm, comic, hilari

Topik 2 berisi tentang: horror, thriller, comedi, scare, scari, laugh, genr, funni, suspens, action

Topik 3 berisi tentang: anim, action, kid, sequel, horror, adventur, origin, franchis, children, pixar

Topik 4 berisi tentang: horror, anim, scare, scari, music, famili, kid, creepi, classic, haunt

Topik 5 berisi tentang: thriller, anim, suspens, comedi, twist, kid, polit, crime, famili, pixar

Topik 6 berisi tentang: romant, music, love, romanc, teen, charm, girl, predict, action, clich

Topik 7 berisi tentang: music, thriller, bond, danc, song, rock, number, western, biopic, scorses

Topik 8 berisi tentang: bond, romant, western, moor, jame, horror, comedi, anim, action, spi

Topik 9 berisi tentang: bond, sequel, famili, kid, seri, drama, franchis, predecessor, origin, jame

Topik 10 berisi tentang: action, war, famili, horror, kid, sport, drama, clich, western, fight

```

# memberi nama topik
nama_topik_baru = {
    'Text_Topic_1': 'Topic_Comedy_Humor',
    'Text_Topic_2': 'Topic_Horror_Thriller',
    'Text_Topic_3': 'Topic_Animation_Adventure',
    'Text_Topic_4': 'Topic_Classic_Creepy',
    'Text_Topic_5': 'Topic_Suspense_Crime',
    'Text_Topic_6': 'Topic_Romance_Teen',
    'Text_Topic_7': 'Topic_Music_Biopic',
    'Text_Topic_8': 'Topic_Spy_Action',
    'Text_Topic_9': 'Topic_Franchise_Sequel',
    'Text_Topic_10': 'Topic_Action_War'
}

tfidf_df = tfidf_df.rename(columns=nama_topik_baru)
tfidf_df = tfidf_df.filter(like='Topic_')

```

Metode TF-IDF (*Term Frequency - Inverse Document Frequency*) diterapkan pada atribut data yang tidak terstruktur yakni ulasan penonton (review_content). TF-IDF adalah teknik pembobotan yang mengukur pentingnya sebuah kata terhadap frekuensinya dalam sebuah dokumen. Penerapan TF-IDF pada ulasan penonton bertujuan untuk menangkap konteks dan elemen intrinsik film yang tidak terwakili oleh sekadar label sentimen (Fresh/Rotten). Sementara label sentimen mengukur tingkat kepuasan penonton, fitur TF-IDF (seperti kata kunci 'action', 'comedi', 'drama')

mengidentifikasi atribut penjualan utama yang secara langsung mempengaruhi keputusan penonton untuk membeli tiket, terlepas dari kualitas film tersebut.

Karena hasil TF-IDF menghasilkan ribuan fitur yang dapat menyebabkan *overfitting*, diterapkan metode LSA (*Latent Semantic Analysis*). LSA menggunakan algoritma PCA dengan mencari kombinasi linear dari kata-kata yang menangkap variasi data terbesar, seperti kata "*laugh*", "*funny*", "*joke*", dan "*comedy*" yang sering muncul bersamaan akan dipadatkan menjadi satu komponen utama (Topik_Comedy_Humor). Hasilnya dari ribuan kolom kata, data direduksi menjadi 10 komponen yang tetap mempertahankan informasi penting dari ulasan asli, namun jauh lebih ringan untuk diproses oleh model Random Forest.

2. One Hot Encoding

```
import numpy as np

df = pd.read_csv('df_final.csv')

def parse(text):
    if pd.isna(text): return []
    return [x.strip() for x in str(text).split(',')]

cols_to_split = ['genres', 'authors', 'actors', 'directors']

for col in cols_to_split:
    df[col] = df[col].apply(parse)

# tambah kolom (untuk fitur)
df['original_release_date'] = pd.to_datetime(df['original_release_date'])

df['release_year'] = df['original_release_date'].dt.year.astype('Int64')
df['release_month'] = df['original_release_date'].dt.month.astype('Int64')

df['num_actors'] = df['actors'].apply(lambda x: len(x) if isinstance(x, list) else 0)
df['num_genres'] = df['genres'].apply(lambda x: len(x) if isinstance(x, list) else 0)
df['num_authors'] = df['authors'].apply(lambda x: len(x) if isinstance(x, list) else 0)
df['num_directors'] = df['directors'].apply(lambda x: len(x) if isinstance(x, list) else 0)
```

```
from collections import Counter
import calendar

# genre
all_genres = [g for sublist in df['genres'] for g in sublist]
top_genres = [g for g, c in Counter(all_genres).most_common(20)]

for g in top_genres:
    col_name = f'Genre_{g.replace(" ", "_")}'
    df[col_name] = df['genres'].apply(lambda x: 1 if g in x else 0)

# actor
all_cast = [c for sublist in df['actors'] for c in sublist]
top_cast = [c for c, count in Counter(all_cast).most_common(20)]

for c in top_cast:
    safe_name = c.replace(" ", "_")
    df[f'Actor_{safe_name}'] = df['actors'].apply(lambda x: 1 if c in x else 0)

# rating
df['content_rating'] = df['content_rating'].fillna('Unrated')

df = pd.get_dummies(df, columns=['content_rating'], prefix='Rating', dtype=int)

# company
top_companies = df['production_company'].value_counts().head(20).index.tolist()

for comp in top_companies:
    safe_name = str(comp).replace(" ", "_").replace(".", "")
    df[f'Studio_{safe_name}'] = df['production_company'].apply(lambda x: 1 if comp in str(x) else 0)

# month
df['month_name'] = df['release_month'].apply(lambda x: calendar.month_abbr[int(x)] if pd.notna(x) else 'Unknown')
df = pd.get_dummies(df, columns=['month_name'], prefix='Month', dtype=int)

# authors
all_authors = [a for sublist in df['authors'] for a in sublist]
top_authors = [a for a, c in Counter(all_authors).most_common(20)]

for a in top_authors:
    safe_name = a.replace(" ", "_")
    df[f'Author_{safe_name}'] = df['authors'].apply(lambda x: 1 if a in x else 0)
```

One Hot Encoding adalah metode untuk mengonversi variabel kategorikal ke dalam format biner. Metode ini menciptakan kolom baru untuk setiap kategori, dengan angka 1 berarti ada dan 0 berarti tidak ada. Penerapan *One-Hot Encoding* dalam penelitian ini dilakukan dengan dua pendekatan strategis berdasarkan karakteristik atribut:

- a. Kategori Tunggal: Untuk kolom `content_rating` dan `release_month`. Setiap kategori (*PG-13*, *R*, *PG*) pada `content_rating` langsung diubah menjadi kolom sendiri (misal: *Rating_PG-13* bernilai 1 jika sesuai). Dan pada kolom `release_month`, data bulan diekstraksi dari tanggal rilis dan diubah menjadi kolom baru, seperti `Month_Dec`, `Month_Jan` untuk menghindari asumsi yang salah (bahwa bulan Desember lebih "besar" nilainya daripada Januari).
- b. Kategori Jamak (Multi-Label): Untuk kolom `genres`, `production_company`, `actors` dan `authors`. Di mana satu film bisa punya banyak nilai (misal: *Action*, *Adventure*, *Comedy*), data dipisahkan dulu lalu diambil kategori terpopuler (*Top N*) untuk dijadikan kolom baru, untuk mencegah jumlah kolom meledak.

2.2.6 Pemodelan

Dalam tahap pemodelan, penelitian ini menerapkan algoritma Random Forest Regression untuk memprediksi pendapatan film. Mengingat distribusi data pendapatan yang skewed akibat keberadaan outlier, dilakukan eksperimen perbandingan antara penggunaan data asli dan transformasi logaritma. Model dibangun dengan konfigurasi 100 pohon keputusan (`n_estimators=100`) menggunakan 80% data latih. Evaluasi kinerja model dilakukan secara komprehensif menggunakan R-Squared (R^2) untuk mengukur akurasi, serta metrik error Mean Absolute Error (MAE), Mean Squared Error (MSE), dan Root Mean Squared Error (RMSE) untuk menghitung tingkat penyimpangan prediksi. Terakhir, dilakukan analisis Feature Importance guna mengidentifikasi atribut yang paling dominan dalam memengaruhi pendapatan film

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.metrics import mean_squared_error
```

1. Tanpa Log Transformation

```
cols_to_drop = ['movie_title', 'revenue']

X = df.drop(columns=cols_to_drop)
y = df['revenue']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)

r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mse = mean_squared_error(y_test, y_pred)

print(f"HASIL AKHIR (TANPA LOG TRANSFORM)")
print(f"R-Squared Score : {r2:.2f} (Akurasi Penjelasan: {r2*100:.1f}%)")
print(f"Rata-rata Meleset: ${mae:,.2f}")
print(f"Meleset: ${rmse:,.2f}")
print(f"Kuadrat Error: {mse:,.2f}")
```

Melatih model menggunakan data pendapatan asli (raw data) tanpa transformasi.

2. Menggunakan Log Transformation

```
df['revenue_log'] = np.log1p(df['revenue'])
cols_to_drop = ['movie_title', 'revenue', 'revenue_log']

X = df.drop(columns=cols_to_drop, errors='ignore')
X = X.select_dtypes(include=[np.number])
X = X.fillna(0)

y_log = df['revenue_log']

X_train, X_test, y_train_log, y_test_log = train_test_split(X, y_log, test_size=0.2, random_state=42)

rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train_log)

y_pred_log = rf_model.predict(X_test)

y_pred_real = np.expm1(y_pred_log)
y_test_real = np.expm1(y_test_log)

r2 = r2_score(y_test_real, y_pred_real)
mae = mean_absolute_error(y_test_real, y_pred_real)
mse = mean_squared_error(y_test_real, y_pred_real)
rmse = np.sqrt(mse)
```

Melatih model menggunakan data pendapatan yang telah ditransformasi ke dalam skala logaritma natural ($\ln(1+x)$) untuk menormalisasi distribusi data dan meredam efek outlier.

BAB III

HASIL DAN PEMBAHASAN

3.1 Hasil Proses

3.1.1 Hasil Preprocessing

Tahap preprocessing ini dilakukan untuk memastikan data yang digunakan dalam kondisi bersih dan siap dianalisis. Preprocessing mencakup dua jenis data utama yaitu data numerik dan data teks berupa review film.

Pada data numerik dilakukan pemilihan atribut untuk analisis. Atribut seperti judul film dan revenue tetap dipertahankan dalam proses pengolahan data awal namun tidak digunakan pada saat pemodelan. Selanjutnya dilakukan pemeriksaan data dari kemungkinan nilai kosong serta ketidak konsistenan format agar tidak mengganggu proses model. Seluruh fitur numerik kemudian disiapkan dalam satu matriks fitur sebagai input model. Selain data numerik, preprocessing juga dilakukan pada data review film berupa data teks. Proses awalnya dilakukan normalisasi teks dengan merubah seluruh huruf menjadi kecil (lowercase). Selanjutnya dilakukan cleaning data dengan menghapus angka, tanda baca, dan simbol yang tidak memiliki makna. Judul film yang muncul didalam review juga dihapus untuk mencegah kebocoran informasi yang dapat mempengaruhi proses ekstraksi topik.

Tahap berikutnya adalah penghapusan stopwords. Selain menggunakan stopword standar bahasa Inggris, ditambahkan juga stopwords manual yang disesuaikan dengan konteks review film seperti kata movie, film, actor, dan director, yang sering muncul tetapi tidak memberikan informasi yang spesifik. Setelah itu, dilakukan proses stemming menggunakan Snowball Stemmer untuk merubah kata ke bentuk dasarnya sehingga variasi kata dengan makna serupa dapat direpresentasikan secara konsisten.

Hasil akhir dari preprocessing teks berupa kolom baru yang berisi review film dalam bentuk teks bersih dan terstandarisasi. Data ini kemudian digunakan dalam proses feature engineering, khususnya untuk pembentukan topik ulasan yang dijadikan fitur tambahan dalam pemodelan prediksi revenue

3.1.2 Feature Engineering

Pada tahap feature engineering dilakukan pembentukan fitur tambahan dari data review agar informasi yang terkandung di dalam komentar dapat dimanfaatkan oleh model. Tujuannya adalah supaya model tidak hanya belajar dari angka seperti budget atau jumlah vote tetapi juga dari pola umum isi ulasan film.

Langkah pertama yang dilakukan adalah mengubah teks ulasan menjadi bentuk numerik menggunakan metode TF-IDF. Metode ini digunakan untuk melihat kata-kata yang sering muncul dalam suatu ulasan namun tidak terlalu umum diseluruh data. Dengan cara ini, kata yang dianggap lebih penting dalam membedakan satu ulasan dengan ulasan yang lainnya akan memiliki nilai yang lebih besar. Jumlah fitur TF-IDF dibatasi hingga 1000 kata, serta hanya kata yang muncul minimal tiga kali yang digunakan agar fitur yang terbentuk tidak terlalu banyak dan tetap relevan.

Hasil dari TF-IDF masih berupa data berukuran besar sehingga dilakukan penyederhanaan menggunakan PCA. PCA digunakan untuk merangkum pola kata yang sering muncul bersama ke dalam beberapa kelompok. Pada penelitian ini, dibentuk 10 kelompok utama mewakili tema umum dari ulasan film. Setiap kelompok tersebut kemudian dicek dengan melihat kata yang paling sering muncul di dalamnya. Berdasarkan kata-kata tersebut, masing-masing kelompok diberi nama topik yang menggambarkan isi ulasan seperti komedi, horor, aksi, film sekuel, atau petualangan animasi. Penamaan topik dilakukan secara manual agar lebih mudah dipahami dan dijelaskan. Fitur topik yang telah terbentuk kemudian digabungkan dengan data film dan digunakan sebagai fitur tambahan dalam proses pelatihan model. Kolom teks ulasan yang asli sudah tidak digunakan karena sudah diwakili oleh fitur topik yang dihasilkan. Dengan adanya fitur tambahan ini, model diharapkan dapat menangkap pengaruh isi ulasan terhadap pendapatan film, sehingga hasil prediksi yang dihasilkan menjadi lebih baik.

Selanjutnya dilakukan one hot encoding untuk mengubah data kategorikan menjadi bentuk numerik agar dapat digunakan oleh model. Proses ini dilakukan pada beberapa variabel yaitu genre film, aktor, rating konten, studio produksi, bulan rilis, dan penulis film. Untuk genre, aktor, studio, dan penulis, dipilih masing-masing 20 kategori yang paling sering muncul agar jumlah fitur tidak terlalu banyak. Setiap kategori direpresentasikan dalam kolom biner dengan nilai 1 jika terdapat dalam film dan 0 jika tidak. Variabel rating konten yang memiliki nilai kosong terlebih dahulu diisi dengan kategori unrated, kemudian diubah menjadi beberapa kolom biner. Bulan rilis juga diubah menjadi fitur biner untuk melihat pengaruh waktu rilis terhadap revenue. Setelah proses encoding selesai, kolom asli yang masih berbentuk teks atau daftar dihapus dari dataset. Hasil akhirnya adalah dataset yang seluruh fiturnya bersifat numerik dan siap digunakan dalam proses pelatihan dan evaluasi model.

3.1.3 Pembagian Data

Setelah melakukan preprocessing dan feature engineering kemudian dataset dibagi menjadi dua bagian, yaitu data latih dan data uji. Pembagian data dilakukan dengan proporsi 80% data latih dan 20% data uji. Data latih digunakan untuk membangun dan melatih model, sedangkan data uji digunakan untuk mengukur kemampuan model dalam melakukan prediksi pada data yang belum pernah dilihat sebelumnya. Pembagian ini bertujuan untuk menghindari

bias hasil dan memastikan bahwa performa model yang diperoleh benar-benar mencerminkan kemampuan prediksi yang sesungguhnya.

3.1.4 Pembentukan Model

Model yang digunakan dalam penelitian ini adalah Random Forest Regressor. Model dibangun dengan membentuk sejumlah pohon keputusan yang dilatih dengan data latih. Setiap pohon menghasilkan prediksi kemudian seluruh prediksi tersebut digabungkan dengan cara dirata-ratakan untuk menghasilkan nilai prediksi akhir. Pendekatan ini membantu mengurangi kesalahan prediksi yang biasanya muncul pada model pohon tunggal. Setelah model terbentuk, proses pelatihan dilakukan menggunakan data latih hingga model siap digunakan memprediksi revenue film pada data uji.

3.2 Evaluasi Model

3.2.1 Uji performa dan Penjelasan

Pada tahap evaluasi, kinerja model Random Forest Regressor diuji menggunakan data uji yang sebelumnya telah dipisahkan dari data latih. Evaluasi dilakukan dengan membandingkan nilai revenue hasil prediksi model dengan revenue sebenarnya.

HASIL AKHIR (TANPA LOG TRANSFORM)
R-Squared Score : 0.75 (Akurasi Penjelasan: 74.6%)
Rata-rata Meleset: \$49,267,340.24
Meleset: \$86,074,791.70
Kuadrat Error: 7,408,869,766,953,464.00

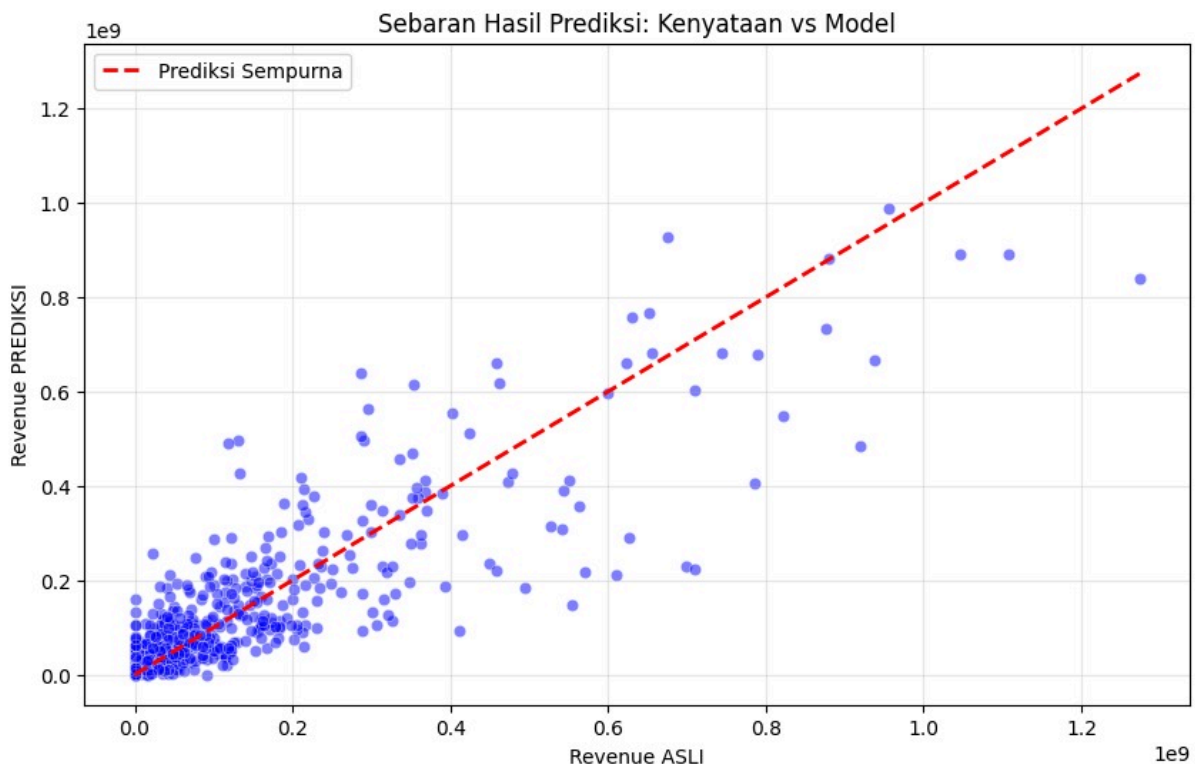
Berdasarkan hasil pengujian, model tanpa log transform menghasilkan nilai R-Squared sebesar 0,75 yang berarti sekitar 74,6% variasi revenue film dapat dijelaskan oleh model. Selain itu, nilai Mean Absolute Error (MAE) menunjukkan rata-rata selisih antara nilai prediksi dan nilai aktual sebesar sekitar 49 juta dolar. Selain itu, nilai Root Mean Squared Error (RMSE) sebesar sekitar 86 juta dolar serta Mean Squared Error (MSE) yang bernilai cukup besar mengindikasikan bahwa masih terdapat beberapa kesalahan prediksi yang cukup tinggi, terutama pada film dengan pendapatan sangat besar. Hasil ini menunjukkan bahwa model mampu memberikan prediksi yang cukup baik terhadap pendapatan film, meskipun akurasi menurun pada nilai revenue yang ekstrem.

Nilai R-Squared yang relatif tinggi menunjukkan bahwa fitur-fitur yang digunakan seperti budget, jumlah vote, popularitas, serta fitur hasil one hot encoding dan ekstraksi topik memiliki kontribusi yang kuat dalam menjelaskan variasi revenue film. Meskipun masih

terdapat selisih prediksi yang cukup besar pada beberapa film, hal ini wajar mengingat nilai revenue memiliki rentang yang sangat lebar terutama pada film-film dengan pendapatan sangat tinggi. Secara umum hasil ini menunjukkan bahwa model Random Forest Regressor tanpa log transform sudah cukup efektif dalam menangkap pola hubungan antara fitur film dan pendapatan yang dihasilkan.

3.2.2 Analisis scatter plot

Scatter plot digunakan untuk membandingkan nilai revenue aktual dengan nilai revenue hasil prediksi model. Garis diagonal merah menunjukkan kondisi prediksi sempurna yaitu ketika nilai prediksi sama dengan nilai sebenarnya.



Berdasarkan visualisasi, sebagian besar titik data berada disekitar garis prediksi sempurna, terutama pada film dengan revenue rendah hingga menengah. Hal ini menunjukkan bahwa model mampu mengikuti pola umum data dan memberikan prediksi yang cukup mendekati nilai sebenarnya. Namun, pada film dengan revenue yang sangat tinggi, terlihat penyebaran titik yang lebih jauh dari garis ideal. Kondisi ini menunjukkan bahwa model masih kesulitan dalam memprediksi film dengan pendapatan ekstrem, sehingga terjadi selisih yang cukup besar antara nilai aktual dan nilai prediksi. Meskipun demikian, secara keseluruhan scatter plot ini menunjukkan bahwa model memiliki kinerja yang cukup baik dan stabil.

3.2.3 Perbandingan Model Log dan Tanpa Log

Perbandingan dilakukan antara model dengan log transform dan tanpa log transform pada variabel revenue.

HASIL AKHIR (DENGAN LOG TRANSFORM)

R-Squared Score : 0.72 (Akurasi: 72.2%)

MAE (Rata-rata Meleset) : \$49,071,875.89

RMSE (Meleset): \$89,935,971.13

MSE (Kuadrat Error): 8,088,478,903,723,997.00

HASIL AKHIR (TANPA LOG TRANSFORM)

R-Squared Score : 0.75 (Akurasi Penjelasan: 74.6%)

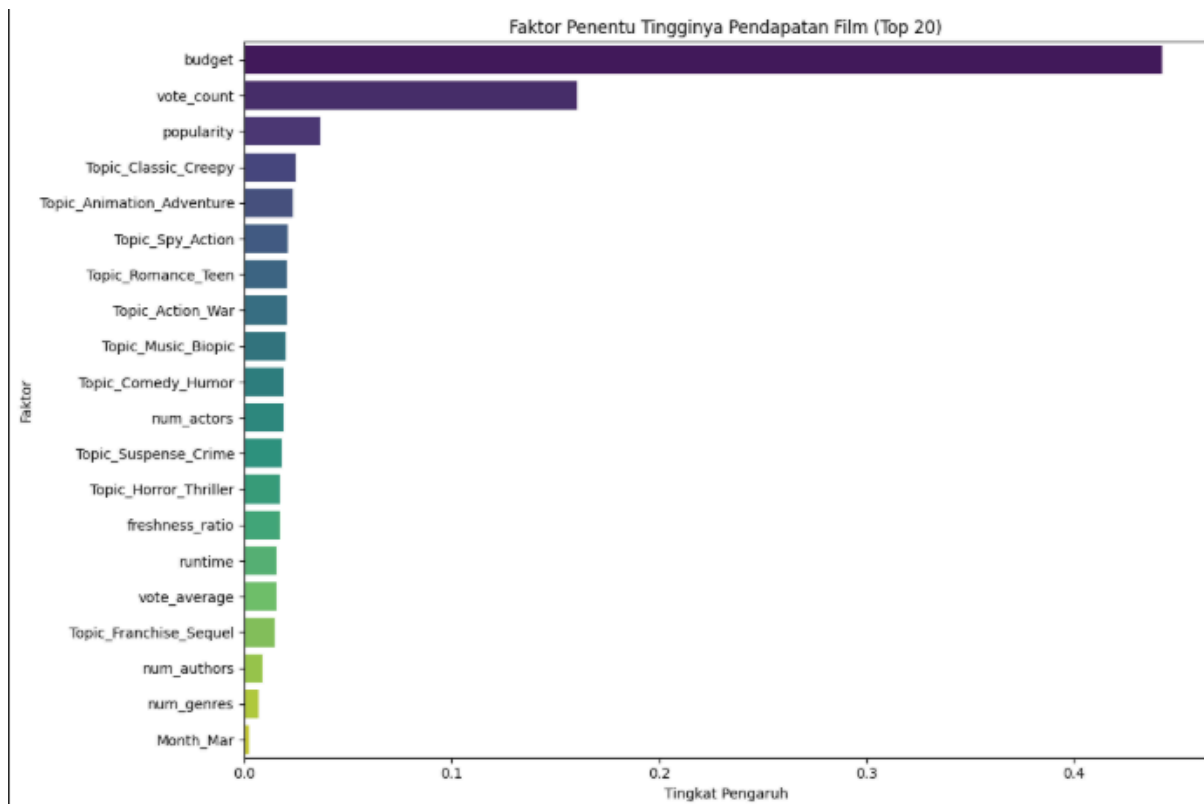
Rata-rata Meleset: \$49,267,340.24

Meleset: \$86,074,791.70

Kuadrat Error: 7,408,869,766,953,464.00

Model tanpa log transform menghasilkan nilai R-Square yang lebih tinggi (0,75) dibandingkan model dengan log transform (0,72), serta MAE yang sedikit lebih rendah. Selain itu, model tanpa log transform juga memiliki nilai RMSE yang lebih kecil, yaitu sekitar 86 juta dolar, dibandingkan model dengan log transform yang mencapai sekitar 90 jkt dolar. Nilai MSE pada model tanpa log transform juga lebih rendah dibandingkan model dengan log transform. Hasil ini menunjukkan bahwa pada dataset yang digunakan, Random Forest Regressor lebih efektif mempelajari pola data revenue asli tanpa perlu transformasi log. Oleh karena itu, model tanpa log transform dipilih sebagai model utama dalam penelitian ini.

3.3 Analisis Faktor Penentu



Semua Faktor Paling Berpengaruh (disortir):

	Fitur	Score
3	vote_count	0.503039
2	budget	0.245128
117	Topic_Franchise_Sequel	0.044527
1	popularity	0.021304
111	Topic_Animation_Adventure	0.015567
0	runtime	0.014841
118	Topic_Action_War	0.012330
109	Topic_Comedy_Humor	0.010507
6	num_actors	0.010495
110	Topic_Horror_Thriller	0.010215
116	Topic_Spy_Action	0.009831
4	vote_average	0.008313
115	Topic_Music_Biopic	0.008223
112	Topic_Classic_Creepy	0.007836
114	Topic_Romance_Teen	0.007745
7	num_genres	0.007281
113	Topic_Suspense_Crime	0.007273
5	freshness_ratio	0.007042
55	Rating_R	0.005615
8	num_authors	0.004946

Berdasarkan analisis feature importance, faktor yang paling berpengaruh terhadap tingginya pendapatan film adalah vote_count dan budget. Hal ini menunjukkan bahwa tingkat keterlibatan penonton serta besarnya anggaran produksi memiliki peran utama dalam menentukan kesuksesan komersial film. Selain itu, fitur bertema seperti Comedy Humor, Action War, dan Animation Advanture menunjukkan bahwa jenis dan tema film turut memengaruhi ketertarikan masyarakat terhadap film. Faktor lain seperti rating penonton, runtime, serta waktu rilis memiliki pengaruh yang kecil namun tetap membantu model dalam menangkap variasi pendapatan film secara keseluruhan.

BAB IV PENUTUP

5.1 Kesimpulan

Berdasarkan hasil penelitian mengenai prediksi pendapatan film menggunakan data historis TMDB dan algoritma Random Forest Regressor, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Faktor Penentu Pendapatan Film

Faktor yang paling berpengaruh terhadap pendapatan film adalah jumlah vote (vote_count) dan anggaran produksi (budget). Selain itu, popularitas film, durasi, jumlah aktor, serta tema atau topik ulasan film juga memberikan kontribusi, meskipun dengan pengaruh lebih kecil. Hal ini menunjukkan bahwa kesuksesan finansial film dipengaruhi oleh kombinasi aspek produksi dan respons penonton.

2. Preprocessing dan Feature Engineering

Proses preprocessing berhasil membersihkan data numerik dan teks, termasuk normalisasi, pembersihan, penghapusan stopwords, dan stemming. Feature engineering melalui TF-IDF dan PCA berhasil mengekstraksi ulasan menjadi fitur topik yang representatif. One hot encoding pada variabel kategorikal memastikan seluruh dataset siap untuk pelatihan model.

3. Performa Model Random Forest Regressor

Perbandingan antara model dengan log transform dan tanpa log transform menunjukkan bahwa model tanpa log transform lebih efektif, dengan R-Squared 0,75 dan MAE lebih rendah dibanding model dengan log transform. Model mampu menjelaskan sebagian besar variasi revenue dan memberikan prediksi yang cukup akurat, terutama untuk film dengan pendapatan rendah hingga menengah. Scatter plot menunjukkan bahwa model kesulitan pada film dengan pendapatan ekstrim, tetapi secara keseluruhan kinerja model cukup baik.

4. Kelayakan Model

Model tanpa log transform yang dihasilkan cukup efektif untuk estimasi awal pendapatan film. Kombinasi data numerik, kategorikal, dan fitur topik ulasan memungkinkan model menangkap pola hubungan antara karakteristik film dan revenue yang dihasilkan.

Dengan demikian, penelitian ini membuktikan bahwa Random Forest Regressor dengan data yang diproses secara tepat dapat digunakan untuk memprediksi pendapatan film dan menganalisis faktor-faktor yang mempengaruhi kesuksesan finansial film, dengan model tanpa log transform sebagai pilihan utama.

5.2 Kontribusi Anggota Kelompok

Nama	NIM	Kontribusi
Ananda Keissa Az Zahra	24031554051	Melakukan tahap preprocessing, EDA dan feature engineering, Laporan BAB 2, PPT
Laili Nurrohmatul Fadhila Z.	24031554093	Laporan BAB 3, Melakukan tahap integrasi data, Mencari dataset, PPT.
Eka Putri Maharani	24031554121	Melakukan tahap modelling, Laporan BAB 1 dan 4, Edit video, PPT

DAFTAR PUSTAKA

- The Movie Database (TMDB). (2017). *TMDB 5000 movie dataset* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata>
- Leone, S. (2020). *Rotten Tomatoes movies and critic reviews dataset* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/stefanoleone992/rotten-tomatoes-movies-and-critic-reviews-dataset>
- CodeSignal. (n.d.). *Customizing TF-IDF vectorization parameters in NLP*. CodeSignal Learn.
<https://codesignal.com/learn/courses/introduction-to-tf-idf-vectorization-in-python/lessons/customizing-tf-idf-vectorization-parameters-in-nlp>
- Darmadi, R. (2023). *Implementasi TF-IDF dan PCA dalam analisis sentimen ulasan film* (Tesis). Universitas Dinamika.
<https://repository.dinamika.ac.id/id/eprint/7259/1/19410100083-2023-UNIVERSITASDINAMIKA.pdf>
- GeeksforGeeks. (n.d.). *One hot encoding in machine learning*. GeeksforGeeks.
<https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/>
- Susanty, Y., Widyastuti, P. P. D., & Dharma, A. (2022). Analysis of the movie database film rating prediction with ensemble learning using random forest regression method. *Jurnal Ilmiah Komputer dan Informatika (KOMPUTA)*, 11(2), 144–150.
<https://jutif.if.unsoed.ac.id/index.php/jurnal/article/view/1563/713>
- Putri, N. R. Y. (2025). *Prediksi pendapatan box office film menggunakan Random Forest berdasarkan anggaran, pemain, dan jumlah penayangan trailer* (Undergraduate thesis). Universitas Islam Negeri Maulana Malik Ibrahim Malang.
<http://etheses.uin-malang.ac.id/75400/>